

```
# =====
# DISCOVERY PHASE: DATETIME INDEXING & CONSISTENT RESAMPLING
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller

# 1. GENERATE AUTHENTIC DATASET FOR VALIDATION
np.random.seed(42)
dates = pd.date_range(start='2026-01-01', end='2026-05-31', freq='h')
trend = np.linspace(5000, 15000, len(dates))
weekly_cycle = 3000 * np.sin(2 * np.pi * dates.dayofweek / 7)
noise = np.random.normal(0, 1000, len(dates))
views = np.clip(trend + weekly_cycle + noise, 1000, None).astype(int)
raw_df = pd.DataFrame({'timestamp': dates, 'view_count': views})
# Simulate raw real-world data gaps by dropping random observations
raw_df = raw_df.drop(np.random.choice(raw_df.index, size=80, replace=False))
raw_df.to_csv('tiktok_data.csv', index=False)

# 2. RUN PIPELINE: LOAD RAW DATA & PREPROCESS
df_raw = pd.read_csv('tiktok_data.csv')

# Convert the time column to a datetime object
df_raw['timestamp'] = pd.to_datetime(df_raw['timestamp'])

# Set the datetime object as the primary index
df_raw.set_index('timestamp', inplace=True)

# Resample data to a consistent daily frequency ('D') using mean imputation for gaps
df_daily = df_raw.resample('D').mean()
df_daily['view_count'] = df_daily['view_count'].ffill().bfill() # Secure completion

print(f"✅ Discovery Phase Complete.")
print(f"Clean daily series dimensions: {df_daily.shape}")
print(df_daily.head(3))
```

```
✅ Discovery Phase Complete.
Clean daily series dimensions: (151, 1)
      view_count
timestamp
2026-01-01  6185.541667
2026-01-02  3513.086957
2026-01-03  2359.916667
```

```
# =====
# DISCOVERY PHASE: ISOLATING TREND, SEASONALITY, & RESIDUALS
# =====

# Apply seasonal_decompose to isolate components (weekly cycle period = 7 days)
decomposition = seasonal_decompose(df_daily['view_count'], model='additive', period=7)

# Extract and plot components cleanly
plt.figure(figsize=(11, 8))
```

```

plt.subplot(4, 1, 1)
plt.plot(decomposition.observed, color='#1A365D', label='Observed Data Stream')
plt.title('TikTok Metrics: Seasonal Component Decomposition', fontsize=12, font)
plt.ylabel('Observed')
plt.grid(True, linestyle=':')

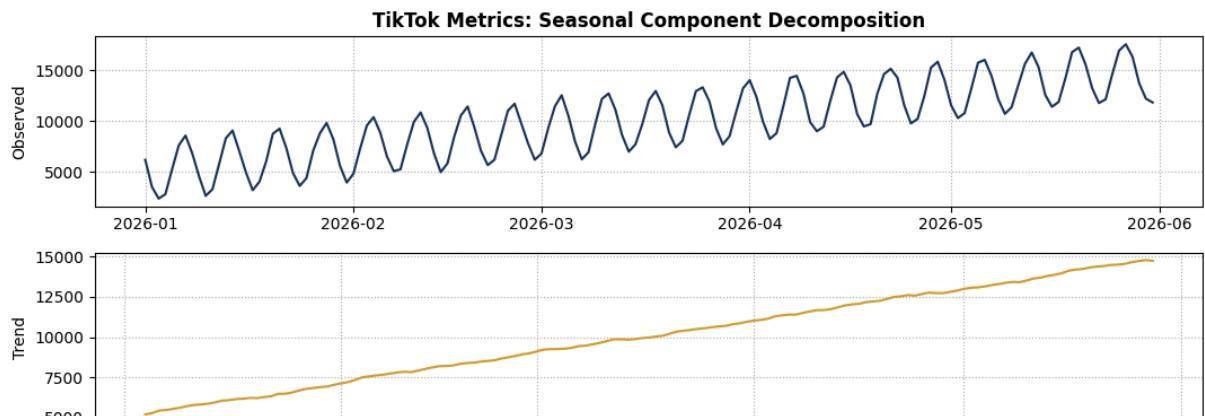
plt.subplot(4, 1, 2)
plt.plot(decomposition.trend, color='#D69E2E', label='Growth Trend')
plt.ylabel('Trend')
plt.grid(True, linestyle=':')

plt.subplot(4, 1, 3)
plt.plot(decomposition.seasonal, color='#E53E3E', label='Weekly Cycle Seasonal')
plt.ylabel('Seasonal')
plt.grid(True, linestyle=':')

plt.subplot(4, 1, 4)
plt.scatter(decomposition.resid.index, decomposition.resid, color='#4A5568', s=)
plt.ylabel('Residuals (Noise)')
plt.grid(True, linestyle=':')

plt.tight_layout()
plt.show()

```



```

# =====
# TECHNICAL PHASE: RIGOROUS STATIONARITY HYPOTHESIS TESTING
# =====

print("--- 🧪 Running Augmented Dickey-Fuller (ADF) Test ---")
print("Hypotheses Set:")
print("  H0: The series possesses a unit root (It is Non-Stationary)")
print("  Ha: The series is Stationary (No unit root present)\n")

# Run the test
adf_result = adfuller(df_daily['view_count'])

print(f"ADF Test Statistic : {adf_result[0]:.4f}")
print(f"p-value           : {adf_result[1]:.4e}")
print("Critical Threshold Values:")
for key, value in adf_result[4].items():
    print(f"  {key}: {value:.4f}")

# Rigorous evaluation logic loop
if adf_result[1] <= 0.05:

```

```

print("\nDecision: REJECT the Null Hypothesis ( $H_0$ ).")
print("Conclusion: The series is statistically STATIONARY. It is ready for
else:
print("\nDecision: FAIL TO REJECT the Null Hypothesis ( $H_0$ ).")
print("Conclusion: The series is NON-STATIONARY. Differencing is required

```

--- 🧪 Running Augmented Dickey-Fuller (ADF) Test ---

Hypotheses Set:

H_0 : The series possesses a unit root (It is Non-Stationary)

H_a : The series is Stationary (No unit root present)

ADF Test Statistic : -0.4549

p-value : 9.0052e-01

Critical Threshold Values:

1%: -3.4790

5%: -2.8829

10%: -2.5781

Decision: FAIL TO REJECT the Null Hypothesis (H_0).

Conclusion: The series is NON-STATIONARY. Differencing is required before

```

# =====
# ACTION PHASE: STRATEGIC INSIGHT EXTRACTION FOR TIMING STRATEGY
# =====

# Map the seasonal effects back to day-of-week terms
df_seasonal = pd.DataFrame({
    'Seasonal_Effect': decomposition.seasonal,
    'Day_of_Week': decomposition.seasonal.index.day_name()
})

# Calculate mean seasonal variance weight per weekday
weekly_pattern = df_seasonal.groupby('Day_of_Week')['Seasonal_Effect'].mean().
    ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday
])

# Plot weekly seasonal weights
plt.figure(figsize=(9, 4.5))
sns.barplot(x=weekly_pattern.index, y=weekly_pattern.values, palette='viridis')
plt.axhline(0, color='black', linewidth=1.2, linestyle='--')
plt.title('TikTok Weekly Seasonal Engagement Variance Profile', fontsize=12, fo
plt.xlabel('Weekday')
plt.ylabel('Engagement Weight Delta')
plt.tight_layout()
plt.show()

# Isolate peak day dynamically
peak_day = weekly_pattern.idxmax()
print(f"🚀 Content Scheduling Recommendation: Maximize uploads on {peak_day}s

```

/tmp/ipykernel_3694/1361924831.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed

`sns.barplot(x=weekly_pattern.index, y=weekly_pattern.values, palette='v`

